

MapReduce Example: World Temperature

(with combiners)

Mahmoud Parsian

Ph.D. in Computer Science

Where the Theory Already Lives

This deck is entirely about one worked trace — a combiner correctly computing an average across multiple partitions. For the *why* behind it (associativity, commutativity, the safe-vs-unsafe aggregate catalog), see:

- [07_combiners_in_mapreduce.md](#)
- [associativity_and_commutativity/Associativity_Commutativity_and_Reducers.md](#)

Same problem as

[08_mapreduce_example_without_combiners.md](#)

— read that one first.

The Pitfall: Average of an Average $\neq$ Average

Two partitions: `{6, 7}` and `{8}` .

```
avg({6,7})      = 6.5  
avg({8})        = 8.0  
avg(6.5, 8.0)  = 7.25          <- WRONG  
  
avg({6, 7, 8}) = (6+7+8)/3 = 7.0  <- correct
```

A combiner that naively pre-averages each partition, then averages *those* averages, gets the wrong answer — because `AVG` is not associative.

The Fix: Delay the Division

Change what the mapper emits: instead of a raw temperature, emit a **(sum, count)** tuple. **SUM** and **COUNT** are both associative and commutative, so combining **(sum, count)** pairs is always safe — *only the final `reduce()` actually divides.*

```
(K, 6) -> (K, (6, 1))  
(K, 7) -> (K, (7, 1))  
(K, 8) -> (K, (8, 1))
```

```
combine: (K, (6, 1)) + (K, (7, 1)) -> (K, (13, 2))
```

```
reduce:  (K, (13, 2)) + (K, (8, 1)) ->  
         (K, (21, 3)) ->  
         (K, 21/3) -> (K, 7.0) ✓
```

New Mapper, Combiner, and Reducer

```
def map(key, value):
    country, city, temperature = value.split(",")
    new_key = f"{country},{city}"
    new_value = (temperature, 1)           # (sum, count), not a raw value
    emit(new_key, new_value)
    emit(country, new_value)

def combine(key, values):                  # values: [(sum,count), ...]
    total = sum(v[0] for v in values)
    count = sum(v[1] for v in values)
    emit(key, (total, count))              # do NOT divide yet

def reduce(key, values):                   # same shape as combine's output
    total = sum(v[0] for v in values)
    count = sum(v[1] for v in values)
    emit(key, total / count)               # divide only here
```

Worked Example: Input (3 Partitions)

Partition-1:

USA,Cupertino,58
USA,Cupertino,67
USA,Sunnyvale,88
USA,Sunnyvale,77
USA,Cupertino,78

Partition-2:

INDIA,Mumbai,90
INDIA,Mumbai,96
INDIA,Agra,98
INDIA,Agra,92

Partition-3:

USA,Cupertino,60
USA,Cupertino,80

Same base dataset as

[08_mapreduce_example_without_combiners.md](#),

plus a 3rd partition with two more Cupertino readings — enough to show a combiner running independently on more than one partition.

Step 1: Mapper Output (Per Partition)

Partition-1	Partition-3
("USA,Cupertino", (58,1))	("USA", (58,1)) ("USA,Cupertino", (60,1)) ("USA", (60,1))
("USA,Cupertino", (67,1))	("USA", (67,1)) ("USA,Cupertino", (80,1)) ("USA", (80,1))
("USA,Sunnyvale", (88,1))	("USA", (88,1))
("USA,Sunnyvale", (77,1))	("USA", (77,1))
("USA,Cupertino", (78,1))	("USA", (78,1))

Step 1 (Continued): Mapper Output — Partition-2

Partition-2

```
("INDIA,Mumbai", (90,1)) ("INDIA", (90,1))  
("INDIA,Mumbai", (96,1)) ("INDIA", (96,1))  
("INDIA,Agra", (98,1)) ("INDIA", (98,1))  
("INDIA,Agra", (92,1)) ("INDIA", (92,1))
```


Step 2: Combiner Output (Per Partition)

Each partition's combiner sums locally, **still as (sum, count)** :

Partition-1:

```
("USA,Cupertino", (203, 3))      # 58+67+78, 3 readings
("USA,Sunnyvale", (165, 2))      # 88+77
("USA", (368, 5))                # all 5 USA readings
```

Partition-2:

```
("INDIA,Mumbai", (186, 2))      ("INDIA,Agra", (190, 2))      ("INDIA", (376, 4))
```

Partition-3:

```
("USA,Cupertino", (140, 2))      # 60+80
("USA", (140, 2))
```

Step 3: Sort & Shuffle — Combiner Output as Input

```
("USA,Cupertino", [(203, 3), (140, 2)])  
("USA", [(368, 5), (140, 2)])  
("USA,Sunnyvale", [(165, 2)])  
("INDIA,Mumbai", [(186, 2)])  
("INDIA,Agra", [(190, 2)])  
("INDIA", [(376, 4)])
```

Only **two** (sum, count) pairs reach the reducer for "USA,Cupertino" — not five individual temperatures. That's the network savings a combiner buys.

Step 4: Reducer Output — Final Averages

```
("USA,Cupertino", (203+140)/(3+2)) -> 68.60
("USA", (368+140)/(5+2)) -> 72.57
("USA,Sunnyvale", 165/2) -> 82.50
("INDIA,Mumbai", 186/2) -> 93.00
("INDIA,Agra", 190/2) -> 95.00
("INDIA", 376/4) -> 94.00
```

Compare to

[08_mapreduce_example_without_combiners.md](#) :

Sunnyvale , Mumbai , Agra , and INDIA match exactly — only

Cupertino / USA differ, because this dataset added two more

Cupertino readings. Same correct averages either way.

Try It Yourself

A 4th partition arrives:

```
CANADA,Toronto,29  
CANADA,Toronto,48  
CANADA,Toronto,61
```

Work out this partition's **mapper output**, its **combiner output**, and the final **reducer output** for "CANADA,Toronto" — then check against [08_mapreduce_example_without_combiners.md](#)'s version of the same exercise (no combiner). They should agree.

Try It Yourself: Answer

```
mapper:    (CANADA,Toronto,(29,1)) (CANADA,Toronto,(48,1)) (CANADA,Toronto,(61,1))
combiner:  (CANADA,Toronto,(29+48+61, 3)) -> (CANADA,Toronto,(138, 3))
reducer:   138 / 3 -> 46.0                                     # matches deck 08 ✓
```

Benefits of Combiners

- Act as **mini-reducers**, running locally on each mapper's output.
- Minimize the number of `(key, value)` pairs crossing the network during Sort & Shuffle.
- Make better use of per-partition resources before the shuffle.
- Avoid handing a reducer one giant `(key, [V1, V2, ..., Vn])` list — which, for a very large `n`, risks an out-of-memory error.

The one requirement that makes all of this safe: the combine function must be associative and commutative — see [07_combiners_in_mapreduce.md](#).